Лабораторная работа №5.

Хранение данных. Volumes и Bind Mounts

Тема: Постоянное хранение данных контейнеров и PostgreSQL в Docker

Цель: Освоить именованные Docker Volumes, Bind Mounts и подтвердить сохранность таблиц PostgreSQL после пересоздания контейнера.

Дисциплина: Docker и контейнеризация приложений

---

1. Общие сведения

Лабораторная работа №5. Хранение данных: Volumes и Bind Mounts. Продолжительность — 4 академических часа (180 минут). Среда: Debian 13 с Docker Engine; терминал Linux; интернет для скачивания образов. Работа продолжает проект Docker курса: мы готовим контейнер PostgreSQL для последующего подключения Express API.

2. Тема и цель

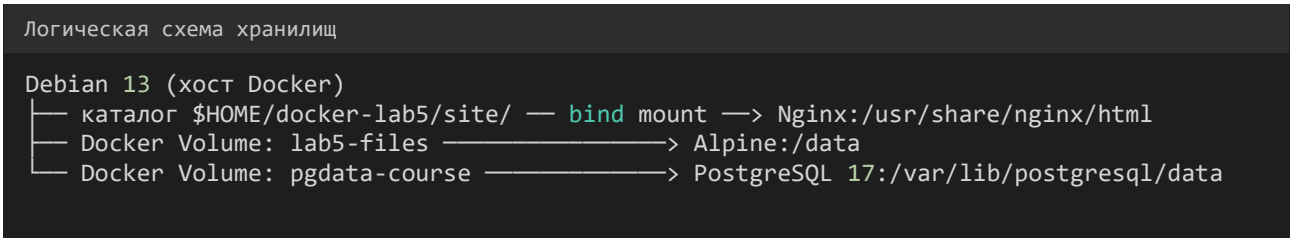
Тема: постоянное хранение файлов контейнера средствами Docker Volumes и Bind Mounts. Цель: понять разницу между записываемым слоем контейнера и внешним хранилищем, создать named volume, исследовать bind mount, развернуть PostgreSQL 17 и подтвердить сохранность записей после удаления и повторного создания контейнера.

3. Оборудование и ПО

- Debian 13 с установленным и запущенным Docker Engine.
- Терминал с доступом к sudo.
- Подключение к Docker Hub для загрузки alpine:3.22, nginx:stable и postgres:17.
- curl для проверки Nginx (проверьте наличие заранее).
- Дополнительная учётная запись Docker Hub не требуется.

4. Схема работы

Логическая схема хранилищ



Каталог хоста и тома создаются на одном Debian-сервере. В этой работе PostgreSQL не публикует порт на хост и доступен для SQL-запросов через docker exec. Имена томов и контейнеров фиксированы, чтобы упростить проверку.

5. Исходные данные

Параметры лабораторной

Объект	Имя / значение
Образ для проверки тома	alpine:3.22
Образ веб-сервера	nginx:stable

Объект	Имя / значение
Образ СУБД	postgres:17
Том для файлов	lab5-files
Том СУБД	pgdata-course
Контейнер веб-сервера	lab5-web
Контейнер СУБД	pg-course
База / пользователь	shop / student
Учебный пароль	Lab5-ChangeMe-2026
Путь тома PostgreSQL	/var/lib/postgresql/data
HTTP порт	8085 хоста → 80 Nginx

Пароль демонстрационный, предназначен только для изолированной учебной среды. Не используйте его для реального сервера. Выбран postgres:17: для этой версии корректный путь тома /var/lib/postgresql/data. Для postgres:18 официальный образ использует иной путь; менять версию самостоятельно не следует.

6. Подготовка среды

Шаг 1. Убедитесь, что Docker установлен и демон доступен.

```
Проверка Docker Engine
sudo docker version
sudo docker info
```

Ожидаемый результат: версии Client и Server, отсутствие ошибки подключения к демону. Если служба остановлена, посмотрите её состояние.

```
Состояние службы
sudo systemctl status docker
```

Если Docker остановлен, запустите службу и повторите docker version.

```
Запуск службы при необходимости
sudo systemctl start docker
```

Шаг 2. Проверка curl; не предполагаем, что программа установлена заранее.

```
Наличие curl
command -v curl
```

При отсутствии пути к исполняемому файлу установите пакет и повторите проверку.

```
Установка curl при необходимости

sudo apt update
sudo apt install -y curl
```

Шаг 3. Скачайте образы.

```
Загрузка учебных образов

sudo docker pull alpine:3.22
sudo docker pull nginx:stable
sudo docker pull postgres:17
```

```
Проверка локальных образов

sudo docker image ls
```

Критерий перехода: все три образа появились в списке. Если возникли ошибки сети или реестра, сообщите преподавателю.

7. Этап 1. Данные внутри контейнера без Volume

Создайте первый временный контейнер и сохраните файл в обычной директории его записываемого слоя.

```
Запись данных в слой контейнера

sudo docker run --name lab5-temp alpine:3.22 sh -c "echo temporary > /tmp/check.txt &&
cat /tmp/check.txt"
```

Ожидается строка temporary; контейнер завершится. Убедитесь, что его можно увидеть в docker ps -a.

```
Контейнер после выполнения программы

sudo docker ps -a
```

Удалите контейнер. Данные из его записываемого слоя больше нельзя получить через этот объект.

```
Удаление временного контейнера

sudo docker rm lab5-temp
```

Создайте другой контейнер из того же образа и проверьте путь.

```
Проверка отсутствия старого файла

sudo docker run --rm alpine:3.22 sh -c "if test -f /tmp/check.txt; then cat
/tmp/check.txt; else echo FILE_NOT_FOUND; fi"
```

```

```

Ожидается FILE_NOT_FOUND. Объясните, почему одинаковый образ не означает общую записываемую файловую систему контейнеров.

8. Этап 2. Создание именованного Docker Volume

Создайте том lab5-files и убедитесь в его наличии.

```
Создание и список томов

sudo docker volume create lab5-files
sudo docker volume ls
```

Ожидается строка с именем lab5-files. Укажите в отчёте имя тома и драйвер, показанный inspect.

```
Метаданные тома

sudo docker volume inspect lab5-files
```

Найдите Name, Driver и Mountpoint. Mountpoint — внутренний путь хранения Docker, не редактируйте его вручную.

Запишите файл в том из первого временного контейнера.

```
Запись в named volume

sudo docker run --rm --mount type=volume,src=lab5-files,dst=/data alpine:3.22 sh -c "echo
persistent > /data/check.txt"
```

Ключ --mount задаёт тип volume, источник lab5-files и путь /data внутри контейнера. Контейнер удалится из-за --rm, именованный том останется.

Прочитайте файл из нового контейнера.

```
Проверка сохранности

sudo docker run --rm --mount type=volume,src=lab5-files,dst=/data alpine:3.22 cat
/data/check.txt
```

Ожидается persistent. Критерий перехода: новый контейнер видит файл, записанный прежним контейнером.

Проверьте доступность тома ещё раз.

```
Повторная проверка тома

sudo docker volume inspect lab5-files
```

Сравните жизненный цикл контейнера и именованного тома. Зафиксируйте наблюдения.

9. Этап 3. Bind Mount

Подготовьте каталог и HTML-файл на Debian.

```
Создание каталога на хосте

mkdir -p "$HOME/docker-lab5/site"
printf "Docker bind mount works\n" > "$HOME/docker-lab5/site/index.html"
```

Каталог и файл создаются обычным пользователем Debian; никаких системных файлов не затрагивается.

```
Проверка содержимого файла

cat "$HOME/docker-lab5/site/index.html"
```

Ожидаемый текст: Docker bind mount works.

Создайте контейнер Nginx с монтированием локального каталога в режиме readonly.

```
Запуск Nginx с Bind Mount

sudo docker run -d --name lab5-web -p 8085:80 --mount type=bind,src="$HOME/docker-
lab5/site",dst=/usr/share/nginx/html,readonly nginx:stable
```

В этой команде src — абсолютный путь к каталогу Debian, dst — каталог сайта в контейнере. readonly запрещает запись из контейнера в примонтированный каталог. Критерий: контейнер появился в docker ps.

```
Проверка HTTP

sudo docker ps
curl http://localhost:8085
```

Ожидается Docker bind mount works. Если веб-сервер не отвечает, изучите docker logs lab5-web и список опубликованных портов.

Измените содержимое файла НА ХОСТЕ и повторите запрос.

```
Эксперимент с обновлением файла

printf "Updated by host\n" > "$HOME/docker-lab5/site/index.html"
curl http://localhost:8085
```

Ожидается Updated by host без пересборки образа и перезапуска контейнера. Сделайте скриншот двух ответов.

```
Проверка Mounts

sudo docker inspect lab5-web
```

В разделе Mounts должны присутствовать Type: bind, Source: каталог Debian, Destination: /usr/share/nginx/html, RW: false.

После проверки остановите и удалите только этот контейнер. Каталог и файл на хосте остаются.

Удаление контейнера и проверка каталога

```
sudo docker stop lab5-web
sudo docker rm lab5-web
cat "$HOME/docker-lab5/site/index.html"
```

Ожидается Updated by host после удаления Nginx.

10. Этап 4. Постоянный том PostgreSQL

Создайте отдельный том для файлов базы данных.

Создание тома PostgreSQL

```
sudo docker volume create pgdata-course
sudo docker volume ls
```

Проверьте наличие pgdata-course. Не используйте этот том для других приложений.

Запустите PostgreSQL 17 с именованным томом и инициализационными переменными.

Первый запуск PostgreSQL

```
sudo docker run -d --name pg-course --mount type=volume,src=pgdata-
course,dst=/var/lib/postgresql/data -e POSTGRES_USER=student -e POSTGRES_PASSWORD=Lab5-
ChangeMe-2026 -e POSTGRES_DB=shop postgres:17
```

Docker создаёт контейнер и подключает pgdata-course по пути данных PostgreSQL 17. Переменные POSTGRES_USER, POSTGRES_PASSWORD и POSTGRES_DB применяются при инициализации пустого каталога. Порт 5432 не публикуется.

Состояние и журнал СУБД

```
sudo docker ps
sudo docker logs pg-course
```

Сразу после запуска служба может ещё инициализироваться. В логах не должно быть постоянной ошибки; дождитесь сообщения о готовности и проверьте pg_isready.

Проверка готовности PostgreSQL

```
sudo docker exec pg-course pg_isready -U student -d shop
```

Ожидается accepting connections. Если получено no response, повторите проверку после завершения инициализации.

11. Этап 5. Создание таблицы и записей

Создайте таблицу categories в учебной базе shop.

Создание таблицы categories

```
sudo docker exec pg-course psql -U student -d shop -c "CREATE TABLE categories (id
INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, name TEXT NOT NULL);"
```

Ожидается CREATE TABLE. Если ранее этот том использовался и таблица уже существует, используйте чистую подготовленную учебную ВМ либо согласуйте очистку с преподавателем: не удаляйте том с чужими данными.

Добавьте две строки.

Добавление тестовых категорий

```
sudo docker exec pg-course psql -U student -d shop -c "INSERT INTO categories (name)
VALUES ('Electronics'), ('Books');"
```

Ожидается INSERT 0 2.

Выполните запрос и сохраните результат.

Проверка данных

```
sudo docker exec pg-course psql -U student -d shop -c "SELECT id, name FROM categories
ORDER BY id;"
```

Ожидаются две категории: Electronics и Books. Если база новая, их ID будут 1 и 2. Сделайте скриншот или скопируйте текст ответа.

12. Этап 6. Проверка после пересоздания контейнера

Остановите и удалите PostgreSQL-контейнер, но не том.

Удаление контейнера PostgreSQL

```
sudo docker stop pg-course
sudo docker rm pg-course
```

Контейнер pg-course исчезнет из docker ps -a. Проверка тома должна показать, что pgdata-course существует.

Проверка сохранности тома

```
sudo docker volume ls
sudo docker volume inspect pgdata-course
```

Критерий продолжения: том pgdata-course существует.

Создайте новый контейнер PostgreSQL 17 с тем же именованным томом и тем же путём монтирования.

Пересоздание контейнера со старым томом

```
sudo docker run -d --name pg-course --mount type=volume,src=pgdata-
course,dst=/var/lib/postgresql/data -e POSTGRES_USER=student -e POSTGRES_PASSWORD=Lab5-
ChangeMe-2026 -e POSTGRES_DB=shop postgres:17
```

При обнаружении существующего кластера PostgreSQL не выполняет первоначальную инициализацию заново. Переменные POSTGRES_* не заменяют автоматически существующих пользователей и паролей.

```
Проверка готовности нового контейнера

sudo docker exec pg-course pg_isready -U student -d shop
```

После accepting connections выполните тот же запрос.

```
Проверка сохранности записей

sudo docker exec pg-course psql -U student -d shop -c "SELECT id, name FROM categories
ORDER BY id;"
```

Ожидается тот же результат: Electronics и Books. Сравните текст вывода до и после пересоздания. Это главный результат лабораторной работы.

13. Диагностические эксперименты

Эксперимент А. Временная остановка сервиса

Остановите контейнер PostgreSQL и попытайтесь проверить готовность базы. Ожидается ошибка выполнения docker exec, потому что контейнер остановлен.

```
Моделирование остановки

sudo docker stop pg-course
sudo docker exec pg-course pg_isready -U student -d shop
```

Восстановите сервис командой docker start; затем дождитесь готовности и убедитесь в наличии категорий.

```
Восстановление после остановки

sudo docker start pg-course
sudo docker exec pg-course pg_isready -U student -d shop
sudo docker exec pg-course psql -U student -d shop -c "SELECT COUNT(*) FROM categories;"
```

Ожидается COUNT = 2. Если PostgreSQL ещё запускается, повторите проверку готовности.

Эксперимент Б. Проверка типа Mount

Проверьте реальную конфигурацию Mounts контейнера и убедитесь, что PostgreSQL использует правильный том.

```
Просмотр подключённых хранилищ

sudo docker inspect pg-course
```

Ожидаемые значения: Type = volume, Name = pgdata-course, Destination = /var/lib/postgresql/data. Сравните с ранее полученным Bind Mount Nginx. Никаких разрушающих операций не выполняйте.

14. Самостоятельное задание

Не удаляя таблицу categories и не используя новые механизмы, самостоятельно добавьте третью категорию Hardware. Затем снова удалите и пересоздайте ТОЛЬКО контейнер PostgreSQL с прежним томом и подтвердите сохранность трёх строк.

1. Добавить категорию Hardware через docker exec и psql.
2. Вывести SELECT до пересоздания и зафиксировать 3 строки.
3. Остановить и удалить только pg-course.
4. Пересоздать pg-course с прежним volume pgdata-course.
5. Проверить pg_isready и повторить SELECT.
6. Зафиксировать Name и Destination из docker inspect.

Ожидается три категории после пересоздания. Не используйте docker volume rm, docker volume prune и docker system prune.

15. Итоговая проверка

```
Итоговая диагностика

sudo docker ps -a
sudo docker volume ls
sudo docker exec pg-course psql -U student -d shop -c "SELECT COUNT(*) FROM categories;"
```

После индивидуального задания ожидаются работающий pg-course, существующий pgdata-course и 3 строки в categories. Именованный том lab5-files также должен присутствовать. Контейнер lab5-web уже удалён.

16. Очистка после защиты

Удалите только временные объекты, которые не понадобятся дальше. PostgreSQL и том pgdata-course рекомендуется сохранить для последующих занятий.

```
Удаление только временного тома

sudo docker volume rm lab5-files
```

Перед выполнением убедитесь, что lab5-files не используется и действительно не содержит нужных данных. Не удаляйте pgdata-course и не применяйте docker volume prune. Каталог $HOME/docker-lab5/site можно оставить как пример Bind Mount.

17. Что должно получиться

- Понимание различий записываемого слоя контейнера, Volume и Bind Mount.
- Созданный и исследованный именованный том lab5-files с проверкой сохранности файла.
- Проверенная связь каталога Debian с Nginx через Bind Mount.
- Запущенный контейнер PostgreSQL 17 с томом pgdata-course.
- Таблица categories со строками, сохранившимися после удаления и пересоздания контейнера.
- Рабочий контейнер pg-course и сохранённый том pgdata-course для будущих лабораторных.

18. Содержание отчёта

7. Тема, цель, исходная среда и версия Docker.

8. Логическая схема Volume / Bind Mount / PostgreSQL.
9. Команды docker volume create, ls, inspect и краткое объяснение.
10. Результат эксперимента с записываемым слоем: FILE_NOT_FOUND.
11. Результат проверки файла в named volume после удаления временного контейнера.
12. Файл index.html на Debian, команда bind mount, вывод curl до и после изменения.
13. Команда запуска PostgreSQL и вывод pg_isready.
14. SQL CREATE TABLE, INSERT и SELECT до пересоздания PostgreSQL.
15. Вывод docker volume ls после docker rm pg-course.
16. SQL SELECT после пересоздания контейнера и в самостоятельном задании.
17. Результаты двух диагностических экспериментов.
18. Вывод: почему Docker Volume подходит для базы данных, а Bind Mount — для исходных файлов.

19. Контрольные вопросы

19. 1. Где Docker сохраняет изменения, если Volume не подключён?
20. 2. Что происходит с записываемым слоем после docker stop и после docker rm?
21. 3. Как работает именованный Docker Volume?
22. 4. Чем Volume отличается от Bind Mount?
23. 5. Для чего применяется параметр readonly у Bind Mount?
24. 6. Что означает --mount type=volume,src=pgdata-course,dst=/var/lib/postgresql/data?
25. 7. Почему для postgres:17 используется именно /var/lib/postgresql/data?
26. 8. Почему нельзя без проверки заменить postgres:17 на postgres:18?
27. 9. Почему прежние данные не исчезли после удаления контейнера?
28. 10. Почему повторное указание POSTGRES_PASSWORD не меняет пароль в существующем кластере?
29. 11. Чем отличается постоянство тома от резервного копирования?
30. 12. Какие поля docker inspect помогают доказать, что PostgreSQL использует нужный том?

20. Критерии оценивания

Оценивание (100 баллов)

Критерий	Баллы
Подготовка и проверка Docker Engine	10
Эксперимент с записываемым слоем	10
Создание и проверка Docker Volume	15
Bind Mount: настройка, HTTP-проверка, inspect	15
Запуск PostgreSQL и SQL-проверка	20
Пересоздание контейнера и сохранность данных	15
Самостоятельное задание и диагностика	10
Отчёт и контрольные вопросы	5
Итого	100

Источники: https://docs.docker.com/engine/storage/volumes/ ;
https://docs.docker.com/engine/storage/bind-mounts/ ; https://hub.docker.com/_/postgres .